TECHNICAL SPECIFICATION & EVALUATION REPORT

Physical Database Design: Primary & Secondary Indexing

Target Workload: Student Database (100,000 Records) **Focus:** Primary Sparse (ID) & Secondary Dense (Pin code) **Status:** Completed Practical Assignment

1. STORAGE ENVIRONMENT & BASELINE PARAMETERS

To establish rigorous quantitative measurements, the baseline relational student storage structure is calibrated against standard fixed-width slotted page allocation constraints:

100,000	128 Bytes	4,096 Bytes	3,125 Blocks
TOTAL STUDENT RECORDS (R)	RECORD SIZE (R)	BLOCK SIZE (B)	DATA FILE FOOTPRINT (B)

Data File Geometry

- **Blocking Factor (Bfr):** $\lfloor 4096 / 128 \rfloor = 32$ records/block
- **Data Block Count (b):** $\lceil 100{,}000 / 32 \rceil = 3{,}125$ blocks
- **Linear Scan Cost:** Average $\lceil b / 2 \rceil = 1{,}563$ block transfers; Worst-case $3{,}125$ I/Os

Index Key Specifications

- **Search Key Field Sizes:** `Student_ID` (8 bytes), `Pin_Code` (4 bytes)
- **Disk Block Pointer (P):** 6 bytes | **Record Pointer (Pr):** 7 bytes
- **Anchor Key Sequencing:** Physically ordered strictly by `Student_ID`

2. STRUCTURAL INDEX BLUEPRINT & SEQUENCING RATIONALE

Primary Index (Attribute: Student_ID)

Design Choice: Primary Ordered Sparse Index

- **Sequencing Match:** The underlying data file is sorted on `Student_ID`. Because records are physically clustered, an index entry for the first record in each data block (block anchor) is sufficient.
- **Entry Size (R_i):** $8\text{ B (Key)} + 6\text{ B (Pointer)} = 14\text{ Bytes}$.
- **Index Blocking Factor (Bfr_i):** $\lfloor 4096 / 14 \rfloor = 292$ entries/block.
- **Index Footprint:** Exactly 1 entry per data block $\rightarrow 3{,}125$ entries total $\rightarrow \lceil 3{,}125 / 292 \rceil = 11$ index blocks.

Secondary Index (Attribute: Pin_Code)

Design Choice: Dense Non-Clustered with Indirect Pointer Buckets

- **Sequencing Mismatch:** The file is NOT sorted by `Pin_Code`. Multiple students share identical postal pins scattered across arbitrary blocks. A sparse index is physically impossible.
- **Pointer Bucket Architecture:** First-level index stores `<Pin_Code, Bucket_Pointer>`. The pointer directs to an overflow block chain containing direct 7-byte record pointers (P_r).
- **Entry Size:** $4\text{ B (Pin)} + 6\text{ B (Ptr)} = 10\text{ B}$. Assuming 500 distinct pin codes: $\lceil 500 / \lfloor 4096/10 \rfloor \rceil = 2$ primary index blocks.

3. QUANTITATIVE SEARCH EFFICIENCY COMPARISON

Search Modality	Target Attribute	Algorithmic Strategy	Block I/O Operations	Execution Latency (10ms I/O)
Unindexed Linear Scan	`Student_ID`	Exhaustive block scan	1,563 (avg) / 3,125 (worst)	15,630 ms (15.63 s)
Unindexed Binary Search	`Student_ID`	File-level binary search	$\lceil \log_2 3125 \rceil = 12$ I/Os	120 ms
Primary Sparse Index	`Student_ID`	Binary search on index + 1 data fetch	$\lceil \log_2 11 \rceil + 1 = 4 + 1 = 5$ I/Os	**50 ms (99.68% speedup vs Scan)**
Unindexed Linear	`Pin_Code`	Full scan (non-key,	3,125 I/Os (must read	31,250 ms (31.25 s)

4. FOUR CORE DATABASE METRICS COMPREHENSIVE EVALUATION

The comparative evaluation between the unindexed base heap/ordered file, primary sparse index, and secondary dense bucket structure reveals predictable performance trade-offs:

Metric 1: Access Time (Read Latency)

Primary Sparse Index: Dramatically reduces equality search retrieval down to $\lceil log_2(b_i) \rceil + 1$ disk transfers. At 11 index blocks, finding an arbitrary `Student_ID` strictly requires **5 block I/Os**.

Secondary Index: Finding all records for a given `Pin_Code` requires $\lceil log_2(b_{sec}) \rceil + 1$ (bucket block read) + m block reads for matching records. For pin codes with 10 students, total cost is ~12 I/Os vs. 3,125 scans.

Metric 2: Insertion Time Overhead

Primary Sparse Index: Minimal maintenance. New records inserted into an existing non-full block do NOT modify the sparse index unless the record has the lowest key in the block, requiring an anchor update. Block splits require 1 index insert.

Secondary Index: Higher overhead. Every single insertion requires appending a record pointer into the corresponding `Pin_Code` bucket list, triggering potential bucket overflows.

Metric 3: Deletion Time Overhead

Primary Sparse Index: Low. Deleting a record only affects the index if that specific record was the block anchor (first key). If so, the second record key becomes the new anchor. Deleting a full block removes 1 index entry.

Secondary Index: Moderate-to-High. The deletion requires traversing the secondary bucket list for that pin code to purge the corresponding 7-byte pointer, and compacting bucket blocks.

Metric 4: Space Overhead Footprint

Primary Index Overhead: Highly negligible. 11 blocks × $4,096$ bytes = **45,056 Bytes (44 KB)**. This accounts for just **0.35%** of the base data file size (12.8 MB).

Secondary Index Overhead: Moderate. 500 index entries = 2 blocks (8 KB). The bucket array stores 100,000 record pointers ($100,000 \times 7 \text{ B} = 700 \text{ KB}$) = ~171 blocks. Total = **708 KB (~5.5% overhead)**.

5. STRUCTURAL EVALUATION SUMMARY TABLE

Design Metric	Unindexed File	Primary Sparse Index (ID)	Secondary Dense + Bucket (Pin)
Exact-Match Lookup	1,563 I/Os (Linear Scan)	5 I/Os (Deterministic)	2 I/Os to bucket + record dereference
Insertion Complexity	$O(1)$ Append / $O(b)$ Sorted	$O(1)$ to block; rare anchor update	$O(1)$ bucket append + potential re-sort
Deletion Cost	$O(b)$ with tombstoning	$O(1)$ inside block; update anchor if key=first	Search bucket + delete pointer ($O(k)$)
Auxiliary Disk Footprint	0 Bytes (0%)	44 KB (0.35% of 12.8 MB data)	~708 KB (5.5% of 12.8 MB data)
Memory Buffer Fit	Cannot buffer 12.8 MB in small RAM	Entire 44 KB fits permanently in RAM cache	Index (8 KB) fits in cache; buckets on disk

6. OPERATIONAL CONCLUSIONS & DEPLOYMENT RECOMMENDATION

1. **Sparse Indexing for Student_ID:** Because the primary physical file is clustered sequentially on `Student_ID`, the primary sparse index achieves optimal compaction (11 blocks), allowing the entire index file to be permanently memory-resident, effectively reducing data block lookups to a single disk access (1 I/O).

2. **Pointer Buckets for Pin_Code:** The secondary index avoids duplicate key inflation by maintaining unique 4-byte key entries pointing to indirect pointer buckets, minimizing the root level index size while accelerating geo-demographic range and aggregate reporting.